```
$results = $xpath->query('//footer');
print "Found {$results->length} matching node(s)";
```

```
Output:
Found 0 matching node(s)
```

What gives? We just used the `//footer` XPath localization step with `DOMXpath::query()`, and it worked just fine. The problem here is that without any information about the default namespace, `Dom\XPath` doesn't know where to search for the `footer` element. The entire document, from its `html` root downwards, is sat inside the "http://www.w3.org/1999/xhtml" namespace. Fortunately, we can grab that namespace from the (slightly magical) document element and tell `Dom\XPath` about it:

```
$xpath->registerNamespace(
    'ns',
    $dom->documentElement->namespaceURI
);

$results = $xpath->query('//ns:footer');
print "Found {$results->length} matching node(s)" . PHP_EOL;
var_dump($results[0]->textContent);
```

```
Output:
Found 1 matching node(s)
string(11) "Page footer"
```

Here we grab the document element's namespace, which will always have the same value because it is standardized (it's up to you if you want to access it from the document element or just use your own constant string) and register it (with a prefix) on `Dom\Xpath`. Once the XPath object is aware of the namespace, we are able to correctly address the `footer` element. Think of it like XPath trying to find a PHP class called `Footer` in the root namespace, but it was actually declared in the `Ns` namespace, and its correct name is `Ns\Footer`. Registering the default HTML namespace, a bit long-winded, and if you prefer, you can prevent `Dom\HTMLDocument` from default namespacing entirely by passing in the `\Dom\HTML_NO_DEFAULT_NS` constant as an option…